

5SDBD - Mineure SDCI

Software Defined Communication Infrastructure

Projet SDCI 2025/26

Equipe enseignante :

S. Medjiah, S. Yangui, E. Akopyan, C. Chassot

Lien d'accès : <http://tiny.cc/Projet-SDCI>

Plan de la présentation

- Compétences ciblées
- Modalités de travail et d'évaluation
- Objectifs, pré requis et séquençement du projet
- Présentation de l'activité IoT ciblée et de son évolution en 3 phases
- 2 visions travaillables au choix des étudiants :
 - **vision 1** : opérateur de réseau, également opérateur de plateforme (PF) de services
 - **vision 2** : opérateur de PF de service “au dessus du réseau” / OTT (Over The Top [*of the network*])
- Travail demandé et organisation
- Plateforme et outils mis à disposition
- Méthodologie de conception

Compétences ciblées
+
Modalités de travail et d'évaluation

Compétences ciblées (1/4)

A l'issue du projet, l'étudiant sera capable de :

- En termes de savoir :
 - Connaître les solutions d'interconnexion des applications logicielles et des capteurs / actionneurs impliqués dans l'IoT, par le biais d'un middleware distribué
 - Comprendre les points de vue des ≠ acteurs du monde réel impliqués dans un projet de ce type
 - Dév. d'application, Opérateur de l'application, Op. middleware (3d party), Op. réseau
 - **Vision 1 (network)** : Comprendre les concepts attendant à la virtualisation de fonctions de réseau dans un système de communication basé sur un réseau programmable (/ défini par le logiciel)
 - **Vision 2 (OTT)** : Comprendre les concepts “d'opérabilité” (au sens : “ops” de Devops) d'une application cloud-native déployée à l'aide de K8s
 - Connaître le modèle de l'autonomic computing défini par IBM

Compétences ciblées (2/4)

A l'issue du projet, l'étudiant sera capable de :

⇒ Vision 1 (Network)

- En termes de savoir-faire :
 - Utiliser un émulateur de réseau SDN = **ContainterNET** (incluant la description d'une topologie réseau via le langage python) (NB : containernet = mininet++)
 - Utiliser un contrôleur SDN (**Ryu**)
 - Utiliser un "MANO NFV" standardisé (**VIM-EMU**)
 - Développer des mécanismes d'ingénierie de trafic au niveau applicatif (L7)
 - Invoquer une interface REST
 - Utiliser (voire étendre) une application IoT sur la base d'un middleware distribué

Compétences ciblées (3/4)

A l'issue du projet, l'étudiant sera capable de :

⇒ Vision 2 (OTT)

- En termes de savoir-faire :
 - Utiliser des fonctionnalités avancées de K8s (**auto scaling, network services**)
 - Utiliser un plugin pour gérer le plan de controle des applications (**istio**)
 - Développer des mécanismes d'ingénierie de trafic au niveau L7
 - Invoquer une interface REST
 - Utiliser (voire développer) une application IoT sur la base d'un middleware distribué

Compétences ciblées (4/4)

A l'issue du projet, l'étudiant sera capable de :

- En termes de compétence :
 - Architecturer et mettre en oeuvre des solutions tirant partie :
 - **Vision 1 (network)** : des concepts de virtualisation de fonctions de réseau (au sens NFV) et de réseaux pilotables par le logiciel (au sens SDN)
 - **Vision 2 (OTT)** : des concepts de Cloud Native Computing & Networking
 - Appliquer et mettre en oeuvre des éléments du modèle de l'autonomic computing en réponse à une problématique de gestion dynamique de QoS dans une SDCI

Modalités de travail et d'évaluation

- Travail par groupe de 2 étudiants en TD et en TP
- Evaluation sous la forme :
 - Rapport final
 - Soutenance finale incluant démonstration
 - Soutenance intermédiaire
 - Implication individuelle durant les TD / TP

Objectif, pré-requis et séquençement du projet

Objectif, pré-requis et séquencement du projet (1/3)

- **Double objectif**

- Déployer *dynamiquement et de façon transparente des traitements intermédiaires*
 - permettant de répondre aux *besoins fonctionnels et/ou non fonctionnels d'applications distribuées* relevant d'une activité de l'Internet des objets (IoT)
 - en appliquant les concepts et techniques relevant :
 - soit de la virtualisation de fonctions de réseau (**NFV**) et des réseaux pilotables par le logiciel (**SDN**) => **vision network**
 - soit du Cloud native Computing & networking (**k8s** et **Istio**) => **vision OTT**
- Développer une approche de *gestion autonome* de la mise en œuvre des traitements ciblés
 - via le concept de l'*Autonomic Computing (AC)* introduit en préambule du projet

Objectif, pré-requis et séquençement du projet (2/3)

● Pré requis

- Autonomic computing ⇒ Mineure SDCI / DAIS
- Network Softwarization (SDN) ⇒ TC SDBD
- Concepts et techniques logicielles associés à la virtualisation ⇒ TC SDBD
- DevOps ⇒ TC SDBD
- Déploiement automatisé d'infrastructures de services (DAIS) ⇒ Mineure SDCI
- Interconnexion de réseaux ⇒ 4IR
- Introduction aux réseaux ⇒ 3MIC

+

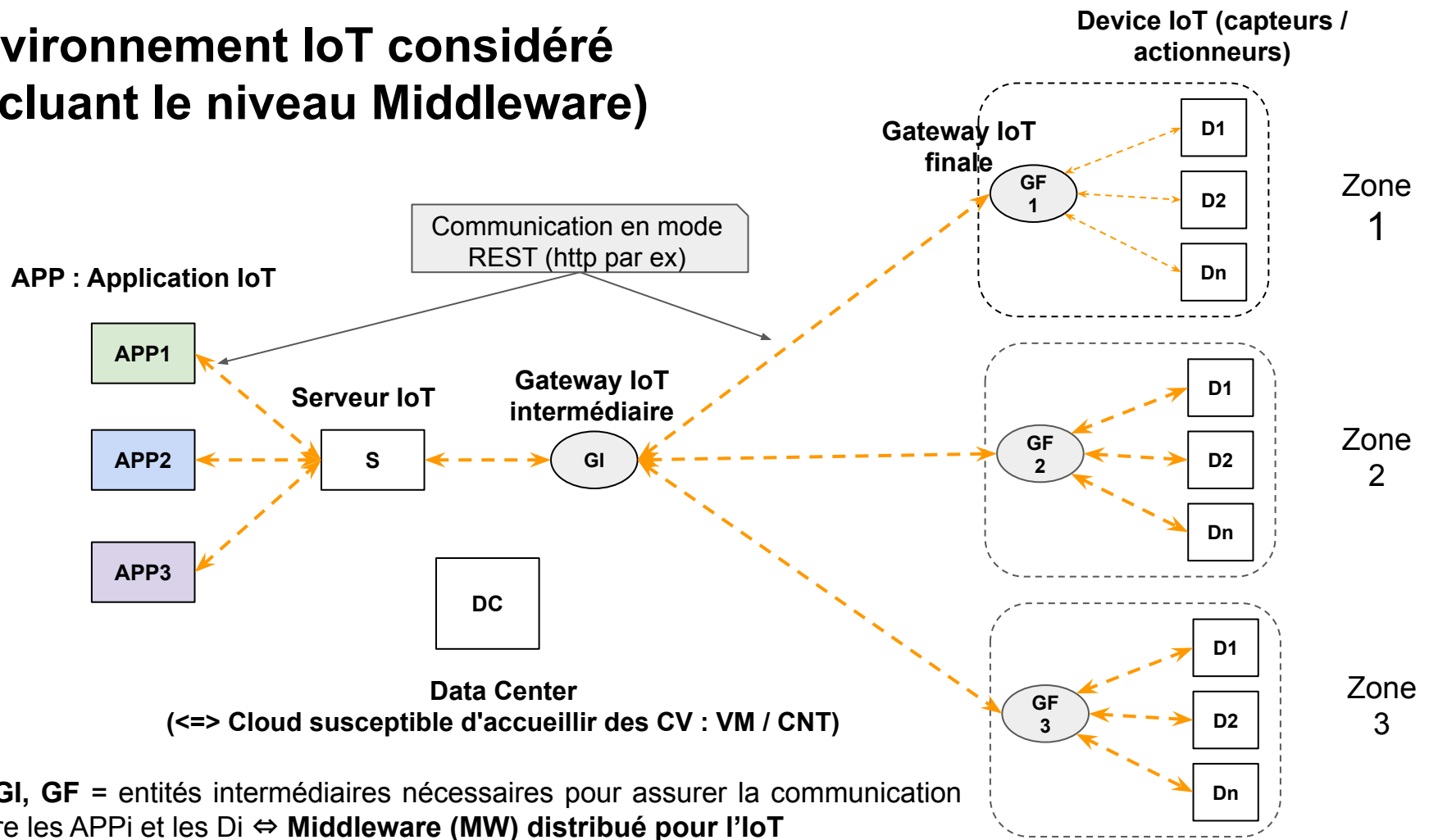
- Architectures logicielles orientées services ⇒ 5SDBD
- Conception orientée objets ⇒ 4IR
- Langage de programmation orientée objet (JAVA, ...) ⇒ 4IR

Objectif, pré-requis et séquençement du projet (3/3)

- **Séquençement prévisionnel du projet (susceptible d'être amendé)**
 - Présentation du projet SDCI
 - 2 séances TD : Conception
 - 2 séances TD : Conception
 - Soutenance intermédiaire (?) : Présentation de la conception par chaque binôme
 - 10 séances TP : Développement
 - Soutenance finale, incluant démonstration
 - Débriefing entre enseignants et étudiants

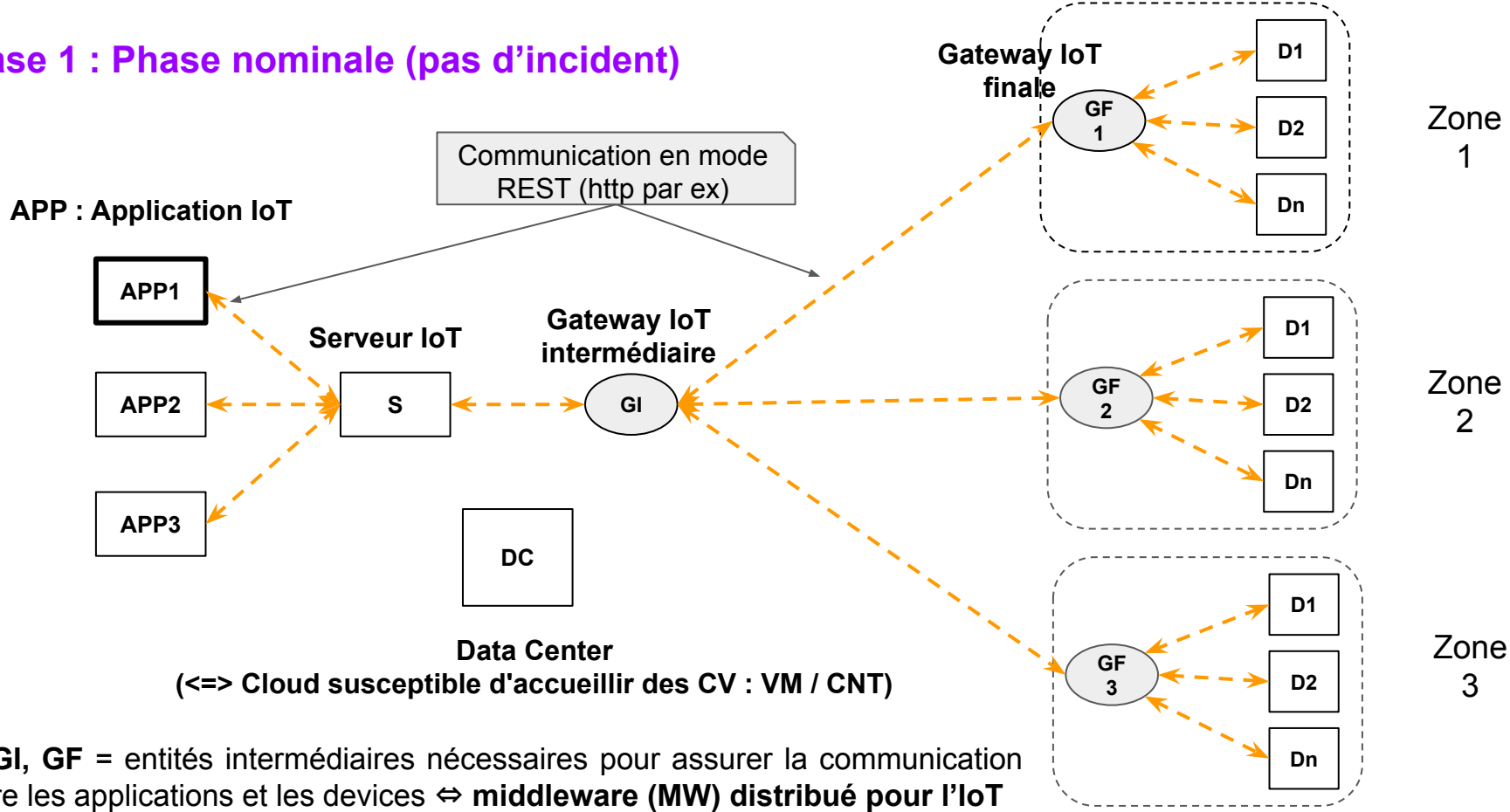
Présentation de l'activité IoT ciblée
et de son évolution en 3 phases
(valable pour visions 1 et 2)

Environnement IoT considéré (incluant le niveau Middleware)



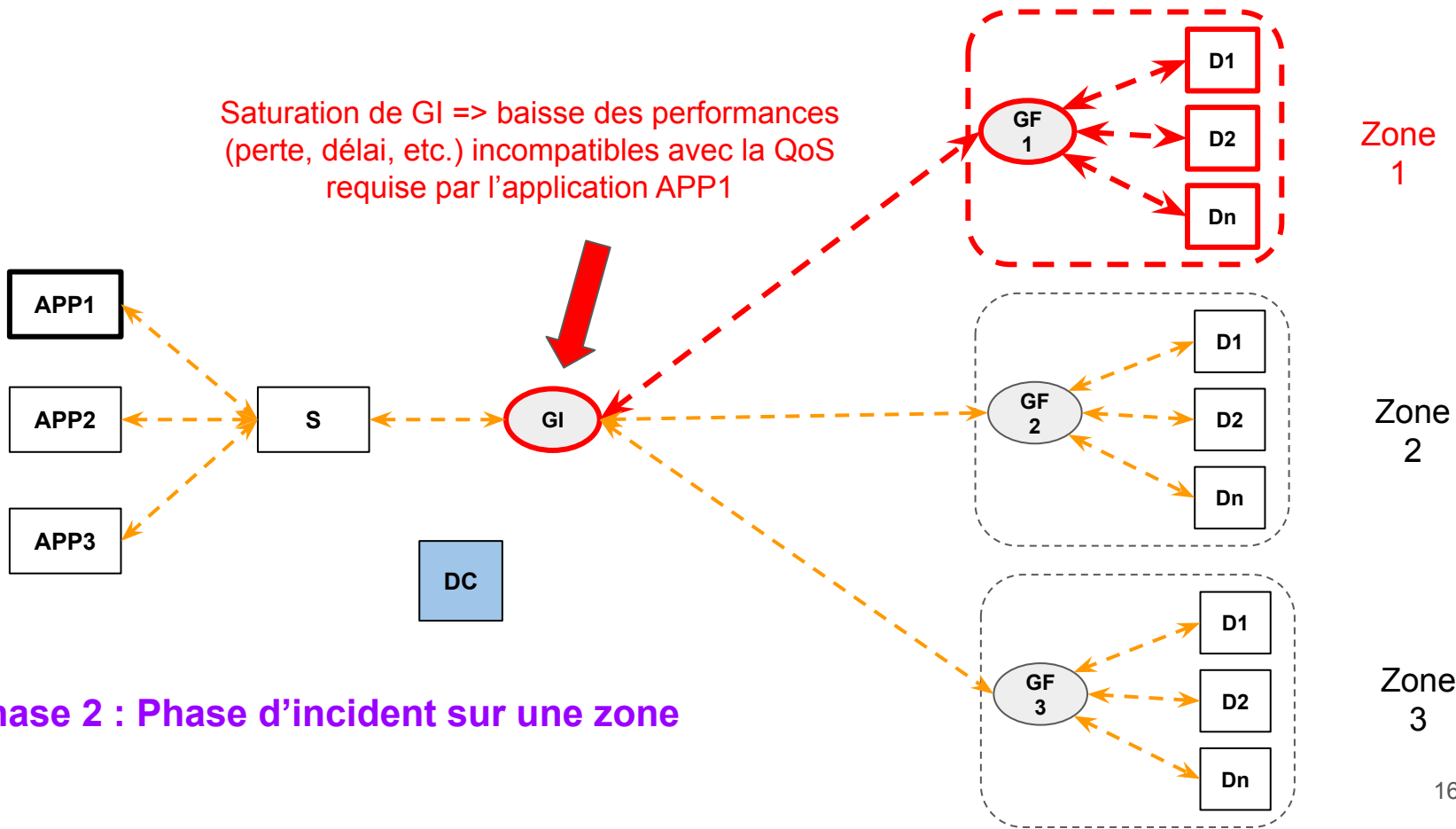
Activité IoT ciblée : Activité de supervision/intervention à distance de ≠ zones dotées de capteurs / actionneurs (Di), par le biais d'une application (APP1)

Phase 1 : Phase nominale (pas d'incident)



Activité IoT ciblée : Activité de supervision/intervention à distance de ≠ zones dotées de capteurs / actionneurs (Di), par le biais d'une application (APP1)

Incident sur la zone 1 => trafic supplémentaire généré par les Di

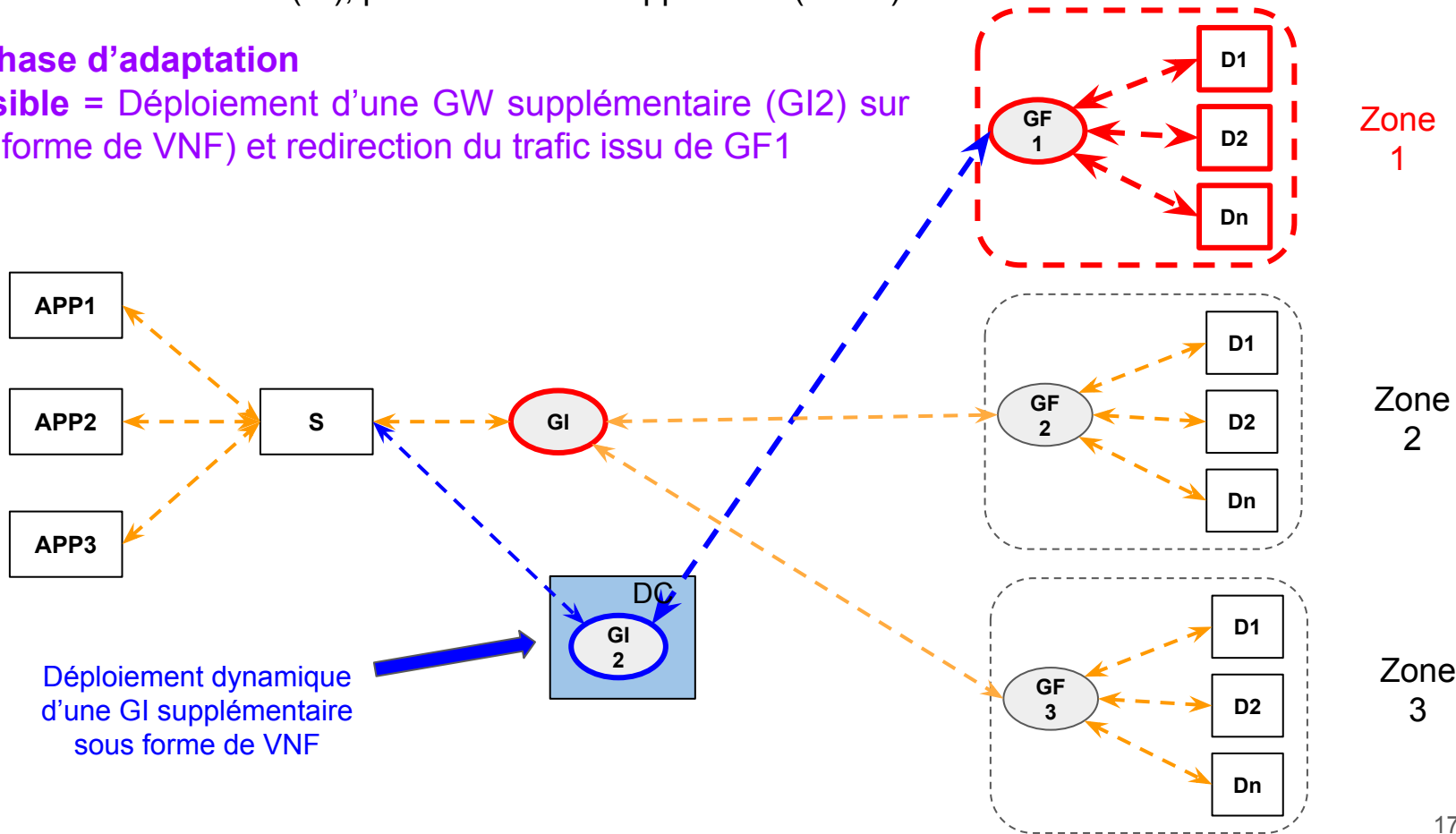


Phase 2 : Phase d'incident sur une zone

Activité IoT ciblée : Activité de supervision/intervention à distance de ≠ zones dotées de capteurs / actionneurs (Di), par le biais d'une application (APP1)

Phase 3 : Phase d'adaptation

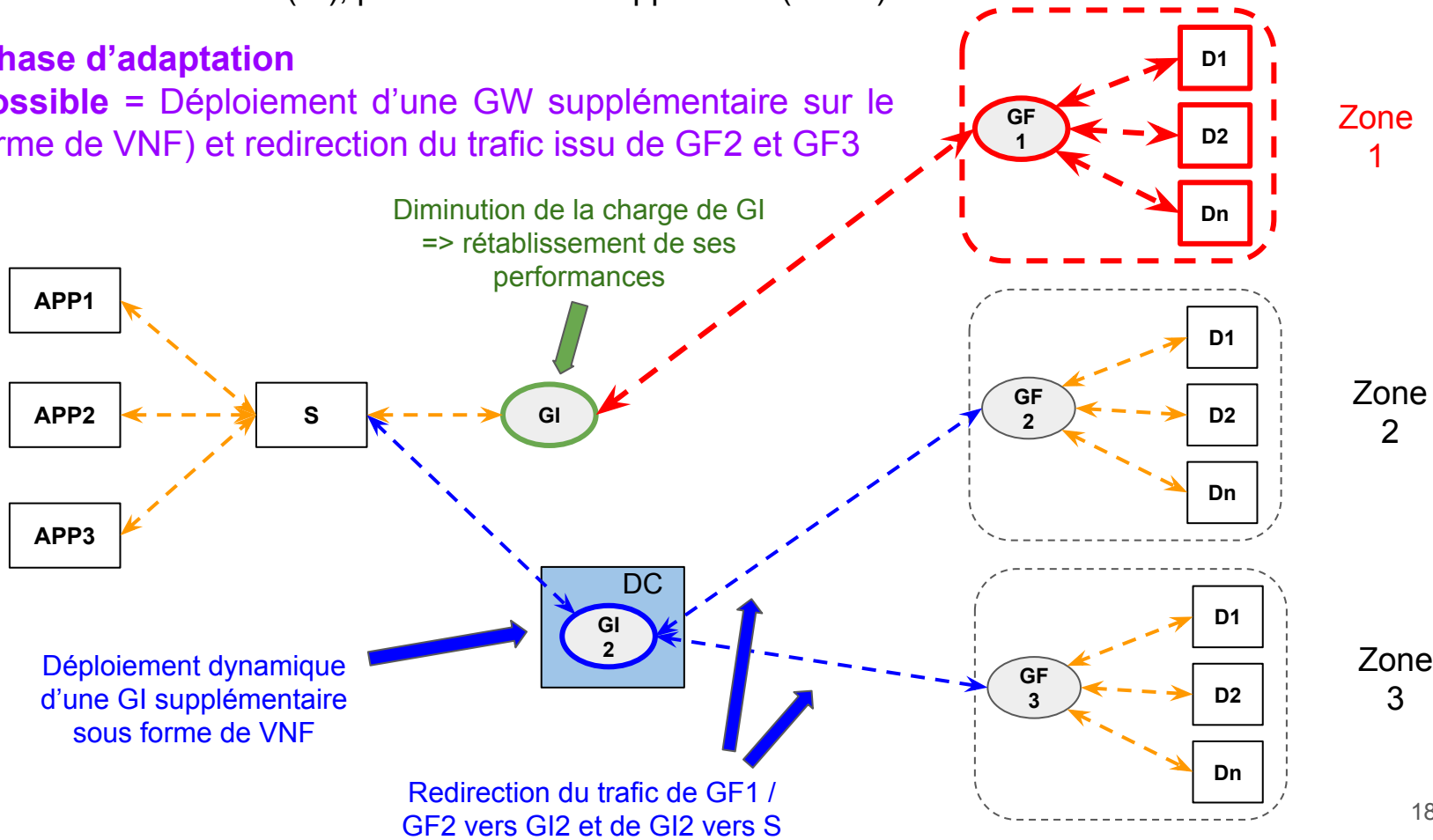
1er ex possible = Déploiement d'une GW supplémentaire (GI2) sur le DC (sous forme de VNF) et redirection du trafic issu de GF1



Activité IoT ciblée : Activité de supervision/intervention à distance de ≠ zones dotées de capteurs / actionneurs (Di), par le biais d'une application (APP1)

Phase 3 : Phase d'adaptation

2eme ex possible = Déploiement d'une GW supplémentaire sur le DC (sous forme de VNF) et redirection du trafic issu de GF2 et GF3

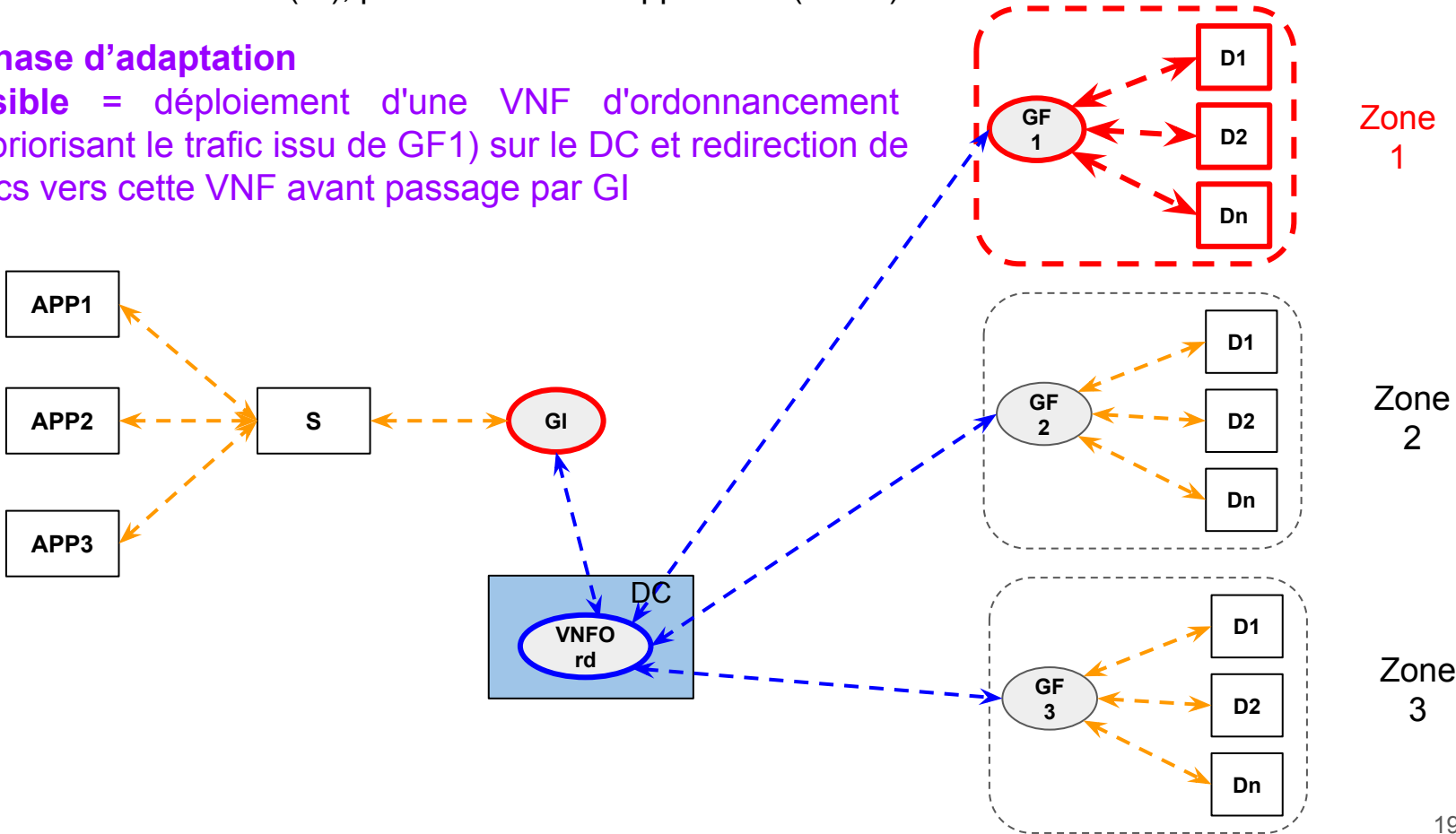


Activité IoT ciblée : Activité de supervision/intervention à distance de ≠ zones dotées de capteurs / actionneurs (Di), par le biais d'une application (APP1)

Phase 3 : Phase d'adaptation

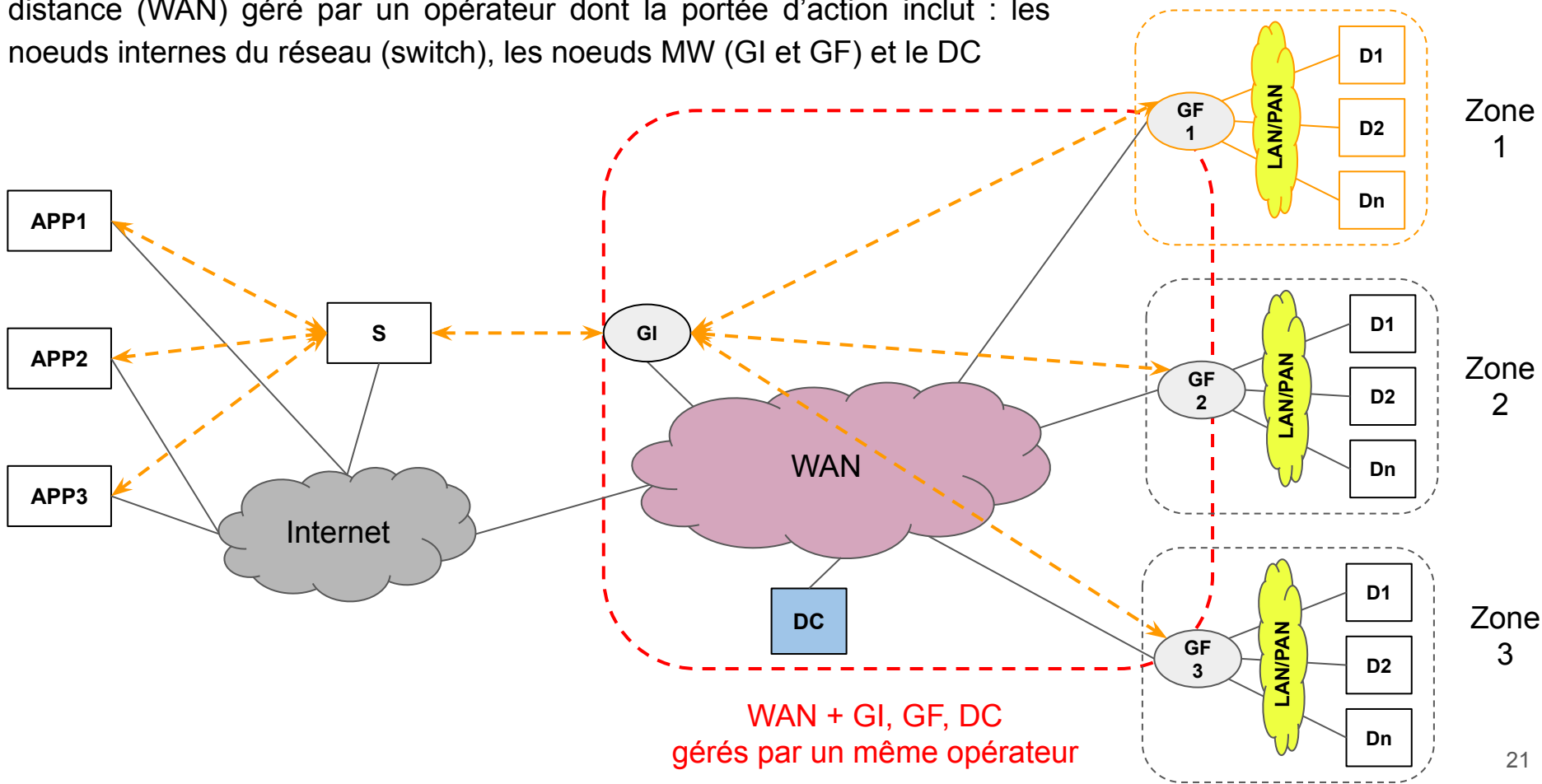
3ème possible = déploiement d'une VNF d'ordonnancement différencié (priorisant le trafic issu de GF1) sur le DC et redirection de tous les trafics vers cette VNF avant passage par GI

Incident => trafic supplémentaire générés par les Di



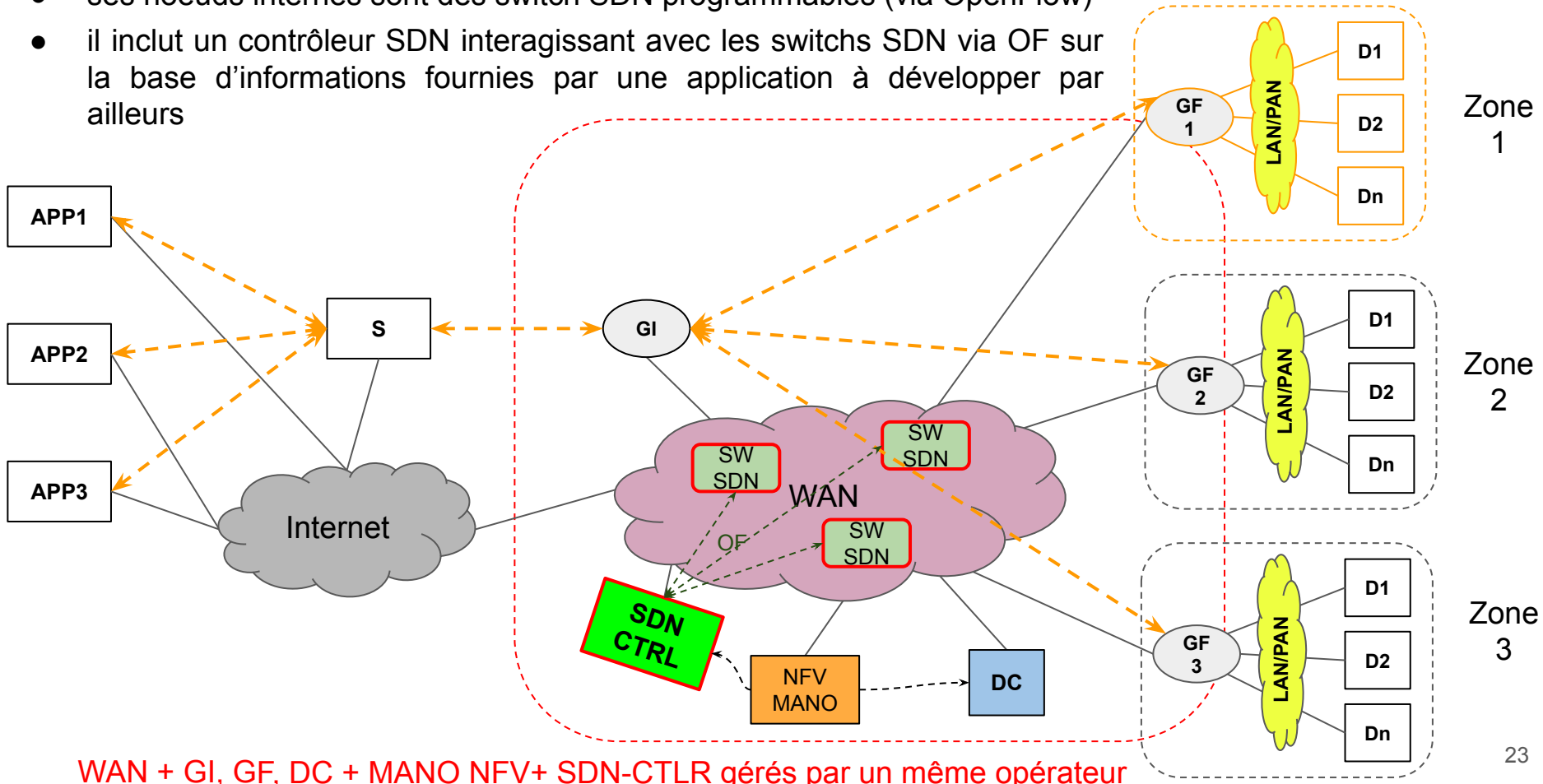
Vision 1 (network) de l'activité ciblée
et de son évolution

HYPOTHÈSE 1 : GI, GF et DC sont connectés via un réseau grande distance (WAN) géré par un opérateur dont la portée d'action inclut : les noeuds internes du réseau (switch), les noeuds MW (GI et GF) et le DC



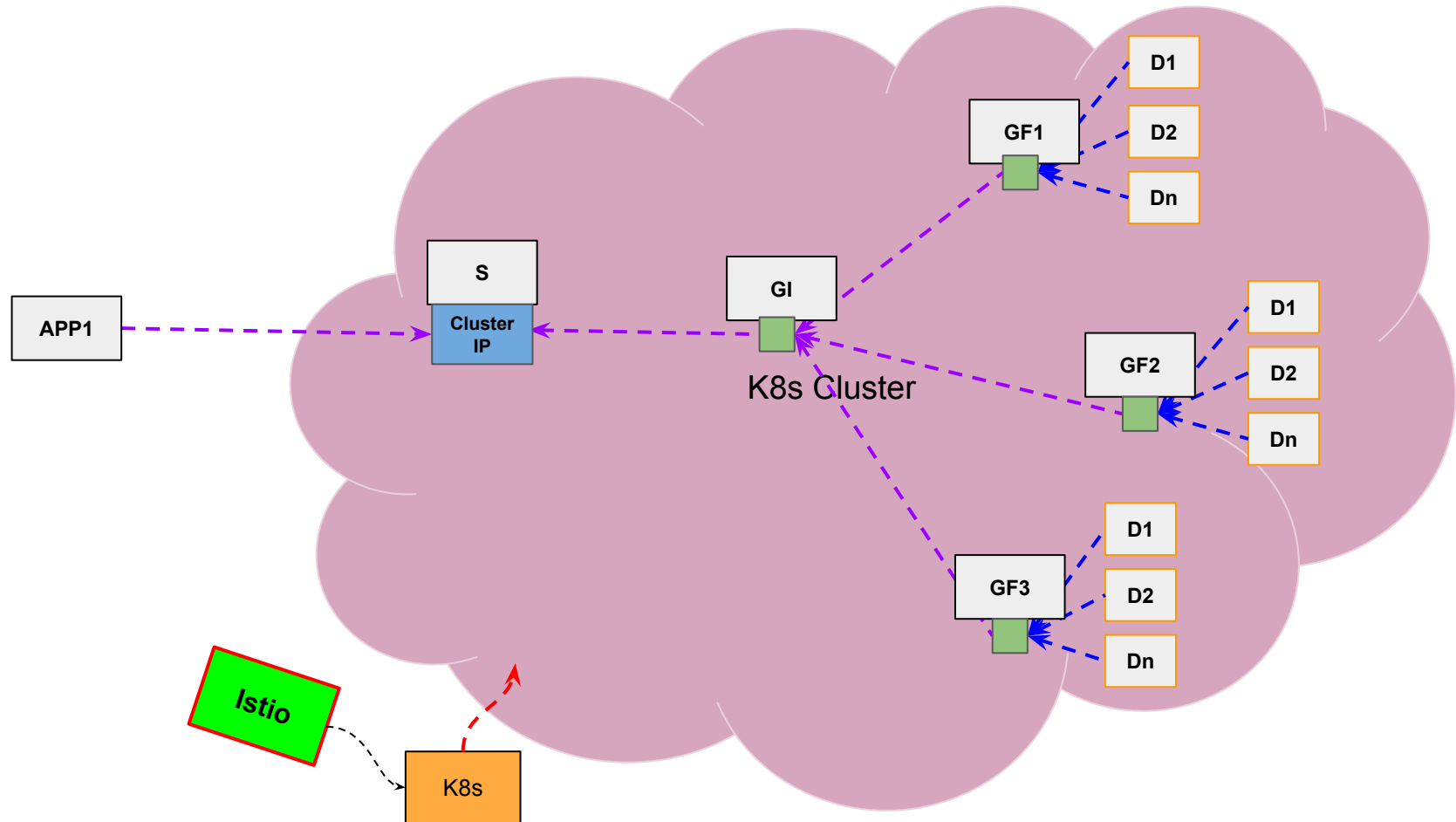
HYPOTHÈSE 3 : Le WAN est doté de capacités SDN, i.e. :

- ses noeuds internes sont des switch SDN programmables (via OpenFlow)
- il inclut un contrôleur SDN interagissant avec les switches SDN via OF sur la base d'informations fournies par une application à développer par ailleurs



Vision 2 (OTT) de l'activité ciblée et de son évolution

HYPOTHÈSE : Cluster K8s avec plugin Istio



Travail demandé et méthodologie

(valable pour visions 1 et 2)

Travail demandé et organisation des TD (1/3)

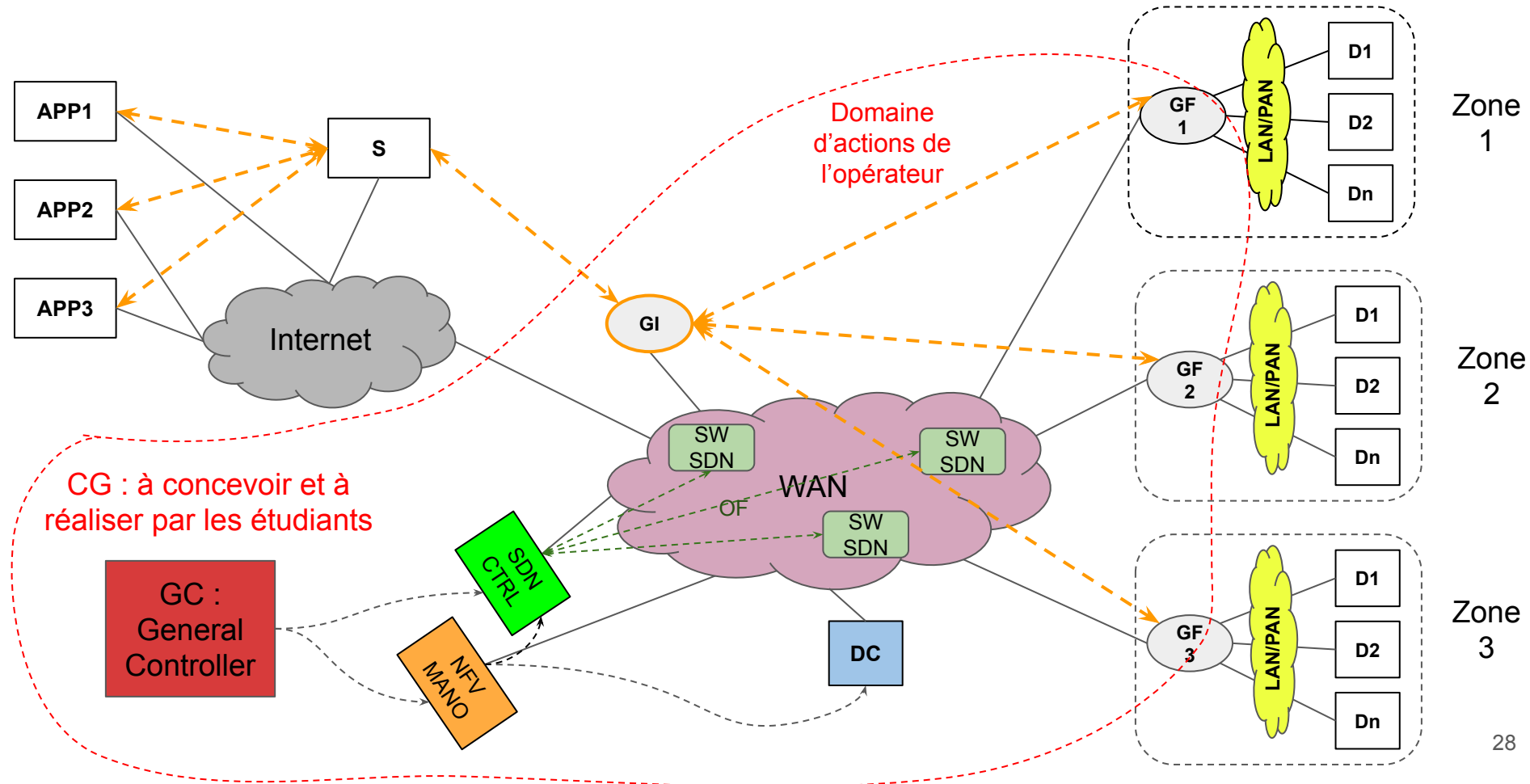
Travail demandé

- Mettre en place l'adaptation requise lorsque l'activité passe de la phase 2 à la phase 3, suivant le cadre de l'Autonomic Computing

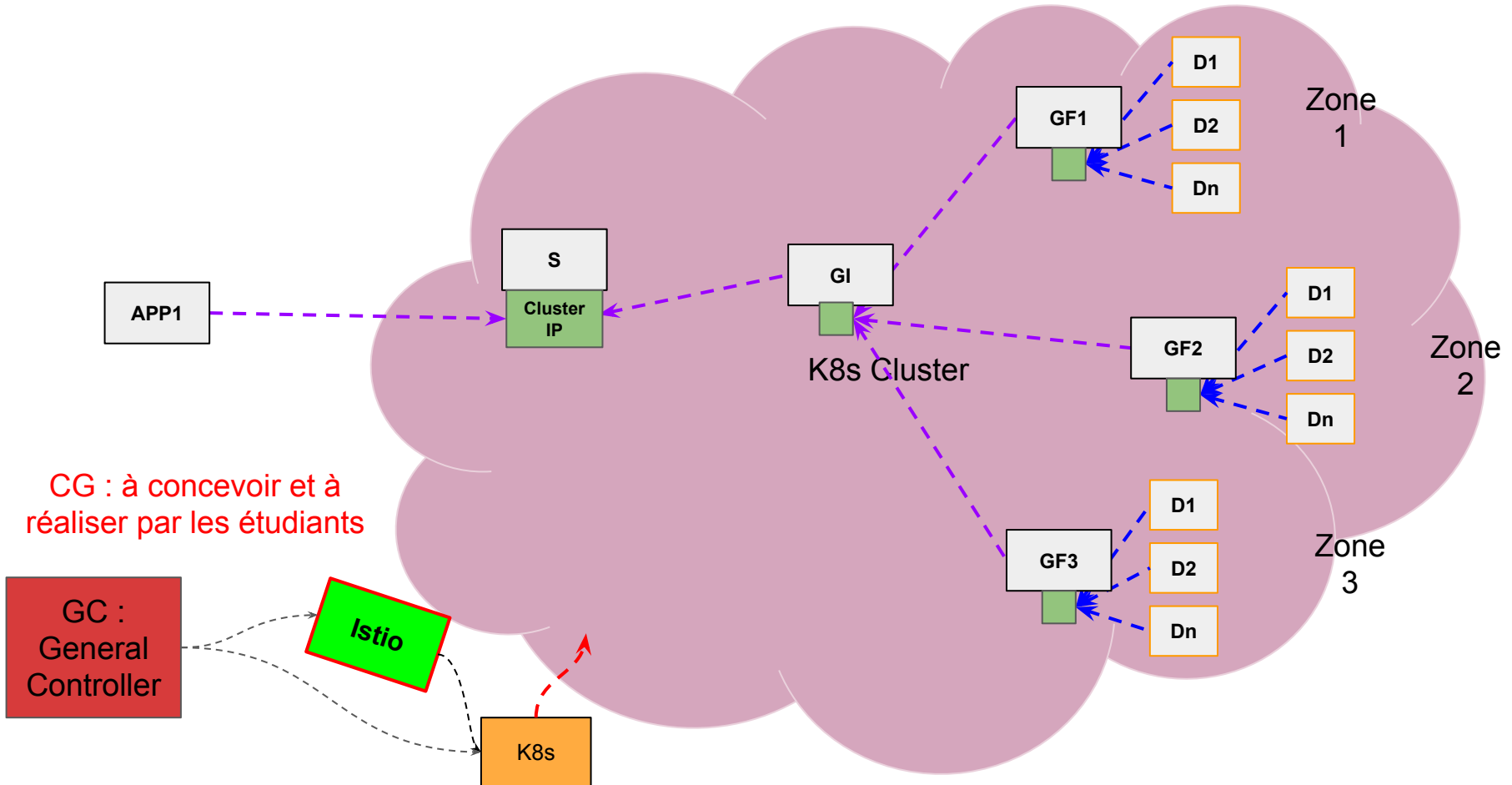
Organisation des TD

- Sur la base du cas d'étude présenté, identifier des mécanismes d'adaptation et les détailler sous la forme de use case (UML) simples
- Proposer une boucle autonome pour l'application de ces mécanismes d'adaptation (Boucle V1)
- Identifier des mécanismes supplémentaires (adaptation ou observation) afin d'enrichir la boucle autonome (Boucle V2)
- Identifier des mécanismes supplémentaires pour améliorer l'analyse et/ou la planification de la boucle autonome (Boucle V3)

Travail demandé (2/3) : Big picture (vision 1 : Network)



Travail demandé (3/3) : Big picture (vision 2 : OTT)



Plateforme et outils mis à disposition (pour la vision 1 Network)

NB : pour la vision 2 :
s'appuyer sur les outils des TP DAIS

Plateforme et outils mis à disposition (1/2)

PF d'émulation de réseau ContainerNet ($\Leftrightarrow$ mininet++) (sous forme d'une VM mise à disposition) incluant :

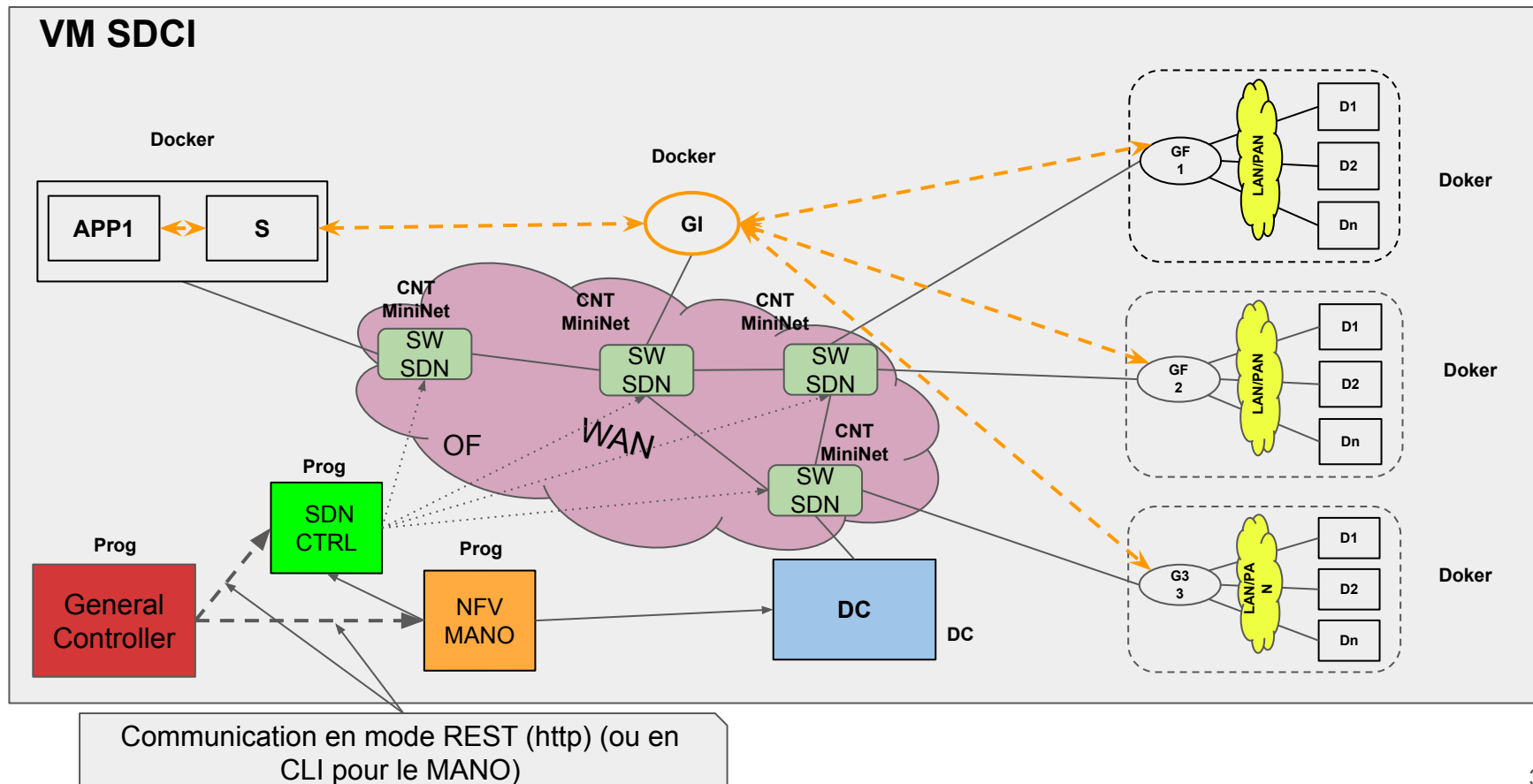
- le programme mn (vu en TP SDN), permettant :
 - d'émuler une topologie de réseau SDN (switchs et hôtes) sous forme de containers LXC ou **Docker**
 - de connecter les switchs à un contrôleur SDN spécifié lors du lancement
- le contrôleur SDN = **Ryu**
 - permettant de contrôler les switchs créés par mn via le protocole OF
 - exposant une API REST pour développer des applications de contrôle
- un MANO standardisé ETSI NFV: **ETSI OSM + son-emu**
 - permettant (via une API REST ou CLI exposées):
 - de déployer des NS et des VNFs en des endroits donnés
 - de lancer / arrêter / (re)configurer ces NS / VNF, ...
 - ...

Plateforme et outils mis à disposition (2/2)

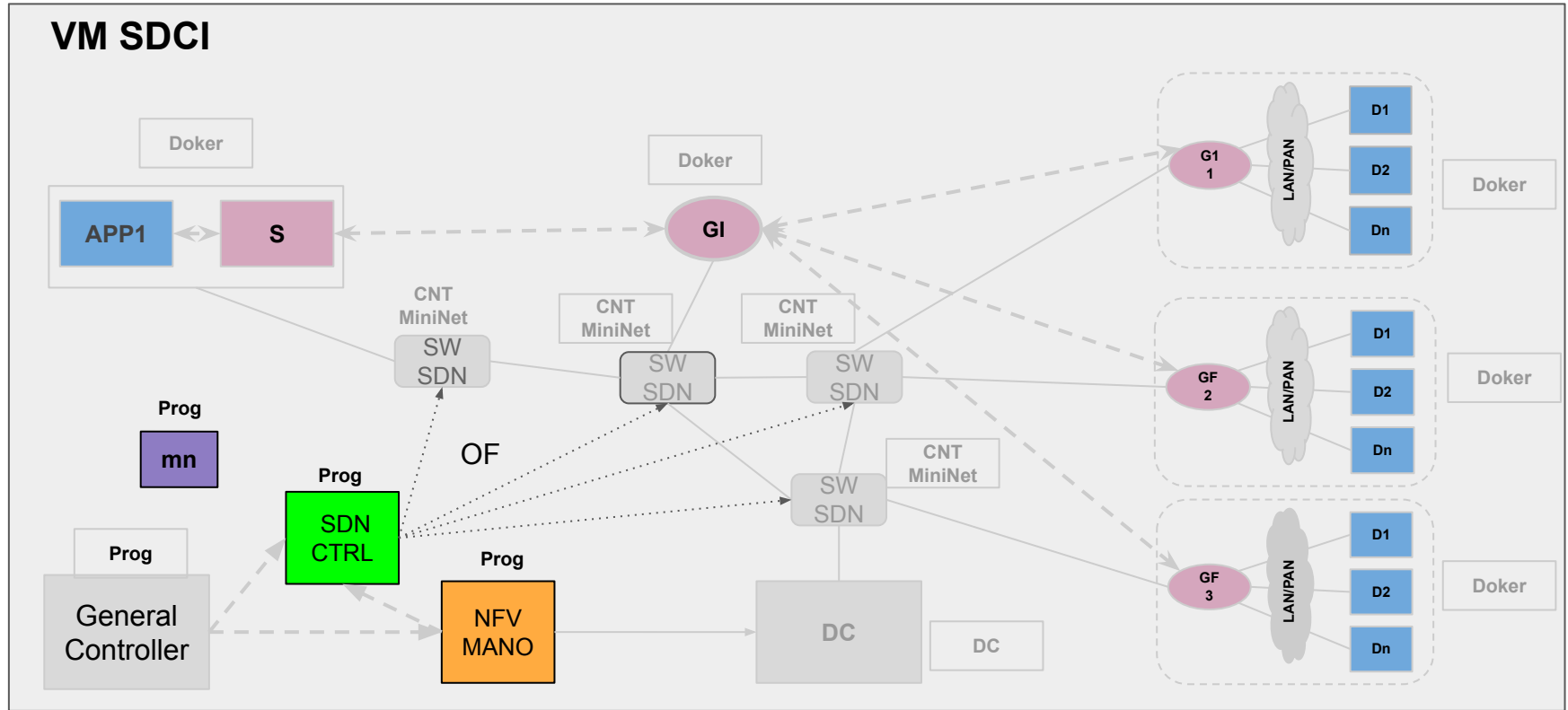
Middleware IoT/M2M = MW en NodeJS (fourni)

- le programme serveur : **iot-server.js**
 - et sa configuration
- le programme gateway : **iot-gateway.js**
 - et la configuration de chaque gateway (finale et intermédiaire)
- un exemple d'une application (en NodeJS) pour émuler un **capteur**
- un exemple d'une application (en NodeJS) pour exécuter les phases de supervision et d'intervention ↔ sous-ensemble de APP1 (à étendre au besoin pour nourrir un scénario complexe)

Big picture des choix d'implantation



Ce qui est mis à disposition des étudiants vs. ce qui est à développer



ContainerNet = programme MiniNet++ (pour la création des noeuds, leur interconnexion, etc.)

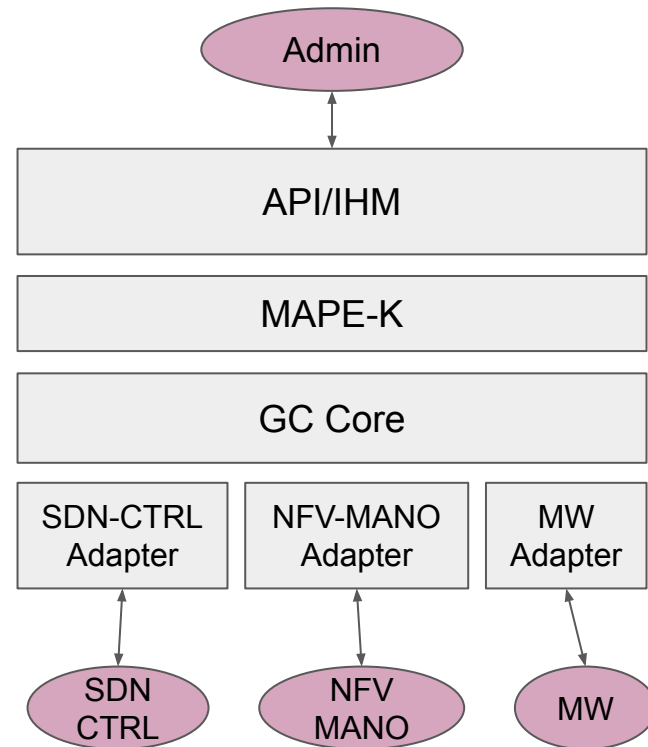
Méthodologie de conception du CG
⇒ basée UML (cf cours COO de 4IR)

Vision 1 (network)

Travail demandé = définir :

- Entités (Acteurs) en interaction avec le GC
 - Administrateur
 - SDN-CTRL, NFV-MANO, noeuds MW (GI, ...)
- Use case (à décrire suivant le canevas fourni)
 - UC 1 :
 - UC 2 :
 - ...
- Diagrammes de structure composite
- Diagrammes de séquence
- Diagrammes de classe (JAVA)

Modèle général du GC (à compléter suivant les UC retenus)

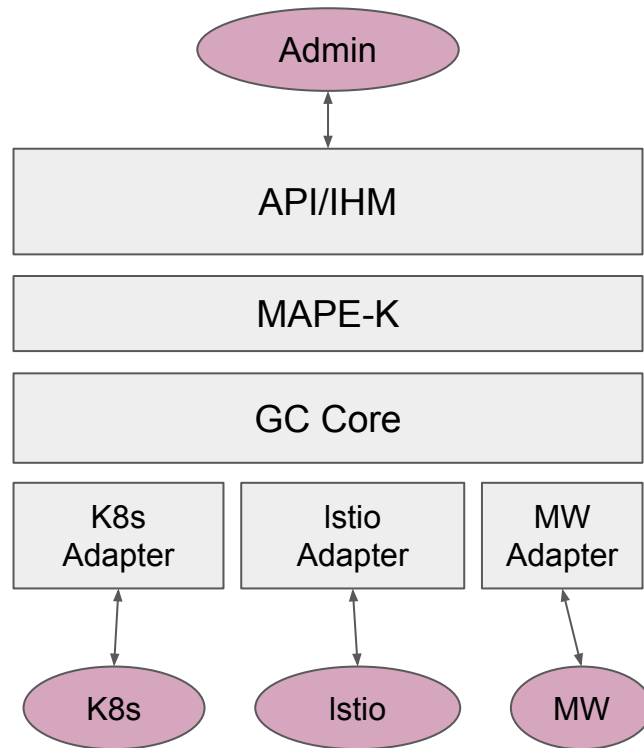


Vision 2 (OTT)

Travail demandé = définir :

- Entités (Acteurs) en interaction avec le GC
 - Administrateur
 - K8s, Istio, noeuds MW (GI, ...)
- Use case (à décrire suivant le canevas fourni)
 - UC 1 :
 - UC 2 :
 - ...
- Diagrammes de structure composite
- Diagrammes de séquence
- Diagrammes de classe (JAVA)

Modèle général du GC (à compléter suivant les UC retenus)



Canevas d'élaboration d'un Use Case

- Use case:
 - Identifiant : *USE_CASE_XXX*
 - Version : *X.Y*
 - Nom : *(...)*
 - Acteurs impliqués : *acteur-1, acteur-2, ..., acteur-N*
 - Description: *(...)*
 - Objectif visé: *(...)*
 - Déclencheurs du UC ($\Leftrightarrow$ action du /des acteurs) : *(...)*
 - Comportement du système en réponse à la demande : *(...)*
 - Pré-conditions: *(...)*
 - Post-conditions: *(...)*

